

Regular Expressions in Python

As a motivation for this topic, say that we have a Fastq file with our data, and we want to check that the adapters were properly removed. The same goes for barcodes and UMIs. How can we remove them from the sequencing data? Regular expressions can do that, and they can also do much more.

In Linux, there are many built-in commands that can process sequencing reads (or any other sequence of letters). However, using regex makes it so that we can process sequencing reads in Python using any system, including windows. Moreover, regex is almost the same in all programming languages, so even Python is not necessary.

Below is a quick explanation about regular expressions, some of it is taken from [here](#), and [here](#). Regular expressions (or 'regex') are essentially a tiny, highly specialized programming language embedded inside Python and made available through the re module.

You can ask questions such as "Does this string match the pattern?", or "Is there a match for the pattern anywhere in this string?". You can also use REs to modify a string or to split it apart in various ways.

Quick example

```
>>> import re
>>> m = re.search('[a-z]+', 'tempo')
>>> m
<re.Match object; span=(0, 5), match='tempo'>
```

The search pattern is [a-z]+, which means search for all letters from a to z.

The square brackets mean 'or', for example [aTg] means a or T or g.

+ means length of 1 and above for the match.

Now we can query the match object for information about the matching string.

```
>>> m.group()
'tempo'
>>> m.start(), m.end()
(0, 5)
>>> m.span()
(0, 5)
```

In addition to the search() method, there are additional methods:

Method/Attribute	Purpose
<code>match()</code>	Determine if the RE matches at the beginning of the string.

<code>search()</code>	Scan through a string, looking for any location where this RE matches.
<code>findall()</code>	Find all substrings where the RE matches, and return them as a list.

Special characters:

`^` Matches the start of the string

`$` Matches the end of the string

`*` Causes the resulting RE to match 0 or more repetitions

`+` Causes the resulting RE to match 1 or more repetitions

`.` this matches any character except a newline

`?` Causes the resulting RE to match 0 or 1 repetitions

`*? , +? , ??` The `'*'`, `'+'`, and `'?'` qualifiers are all greedy; they match as much text as possible. Adding `?` after the qualifier makes it perform the match in non-greedy or minimal fashion; as few characters as possible will be matched.

`{m}` Specifies that exactly `m` copies of the previous RE should be matched

`{m,n}` Causes the resulting RE to match from `m` to `n` repetitions

`|` the choice operator (`'a|b'` means either `a` or `b`)

`\` Either escapes special characters (permitting you to match characters like `'*'`, `'?'`, and so forth), or signals a special sequence

`\w` means a word character `[a-zA-Z0-9_]`

`\W` anything but word character

`\n` end of line

`\t` tab

`[]` Used to indicate a set of characters. In a set: Characters can be listed individually, e.g. `[amk]` will match `'a'`, `'m'`, or `'k'`. Ranges of characters can be indicated by giving two characters and separating them by a `'-'`, for example `[a-z]` will match any lowercase letter, `[0-5][0-9]` will match all the two-digits numbers from 00 to 59. Special characters lose their special meaning inside sets. For

example, `[(+*)]` will match any of the literal characters `'('`, `'+'`, `'*'`, or `')'`. If the first character of the set is `^`, all the characters that are not in the set will be matched.

Examples (from [here](#)):

- `.at` matches any three-character string ending with "at", including "hat", "cat", "bat", "4at", "#at" and " at" (starting with a space).
- `[hc]at` matches "hat" and "cat".
- `[^b]at` matches all strings matched by `.at` except "bat".
- `[^hc]at` matches all strings matched by `.at` other than "hat" and "cat".
- `^[hc]at` matches "hat" and "cat", but only at the beginning of the string or line.
- `[hc]at$` matches "hat" and "cat", but only at the end of the string or line.
- `\[.\\]` matches any single character surrounded by "[" and "]" since the brackets are escaped, for example: "[a]", "[b]", "[7]", "[@]", "[]", and "[]" (bracket space bracket).
- `s.*` matches `s` followed by zero or more characters, for example: "s", "saw", "seed", "s3w96.7", and "s6#h%(>>>m n mQ".
- `[hc]?at` matches "at", "hat", and "cat".
- `[hc]*at` matches "at", "hat", "cat", "hhat", "chat", "hcat", "cchchat", and so on.
- `[hc]+at` matches "hat", "cat", "hhat", "chat", "hcat", "cchchat", and so on, but not "at".
- `cat|dog` matches "cat" or "dog".

Lookahead and lookbackwards

`(?=...)`

Matches if ... matches next, but doesn't consume any of the string. This is called a *lookahead assertion*. For example, `Isaac (?=Asimov)` will match 'Isaac ' only if it's followed by 'Asimov'.

`(?!...)`

Matches if ... doesn't match next. This is a *negative lookahead assertion*. For example, `Isaac(?!Asimov)` will match 'Isaac ' only if it's *not* followed by 'Asimov'.

`(?<=...)`

Matches if the current position in the string is preceded by a match for ... that ends at the current position. This is called a *positive lookbehind assertion*. For example:

```
>>> m = re.search('(?<=-)\w+', 'spam-egg')
>>> m.group(0)
'egg'
>>> m = re.search('(?<=-)\w+', 'spam-egg ok-computer')
>>> m
<re.Match object; span=(5, 8), match='egg'>
>>> m = re.findall('(?<=-)\w+', 'spam-egg ok-computer')
>>> m
['egg', 'computer']
```

`(?<!\...)`

Matches if the current position in the string is not preceded by a match for `\...`. This is called a *negative lookbehind assertion*.